# Hosting (MVP launch)

HR Track needs three always-on pieces:

1. **API** — Fastify (REST + WebSocket `/ws` + hourly in-process jobs)
2. **Web** — Next.js PWA
3. **Postgres 16** — primary datastore

Do **not** put the API on sleeping free PaaS or pure serverless. Cold starts break live map WebSockets and can delay Paymob webhooks.

## Recommendation: zero cost vs best

| Path | Cost | What to use | When |
|---|---|---|---|
| **Zero cost** | **$0 forever** | Oracle Cloud Always Free ARM VM (`VM.Standard.A1.Flex`, up to 2 OCPU / 12 GB) + this repo's `docker-compose.prod.yml` + optional Caddy TLS | You want a real always-on MVP with no monthly bill |
| **Best (cheap paid)** | **~$5–20/mo** | Railway Hobby (API + Postgres + web), or API+DB on Railway and web on Vercel | You want the least ops and managed HTTPS |
| **Avoid** | "Free" sleep tiers | Render free, Railway free ($1 credit), serverless API | Sleep + no durable WS + missed webhooks |

This repo ships a **portable production Docker Compose** stack so either path works.

```
Users ──► Caddy (HTTPS) ──► web :3000
                  │
                  └──► api :4000 ──► Postgres
Paymob ──► POST {API_PUBLIC_URL}/v1/webhooks/paymob
```

## Prerequisites

- Docker Engine + Docker Compose v2
- Domains (recommended) or a public IP
- Strong secrets for JWT and Postgres
- For billing: Paymob credentials (see BILLING.md)

## Environment

```
cp .env.production.example .env.production
# edit YOUR_HOST, passwords, JWT secrets, and (for TLS) WEB_DOMAIN /
```

```
API_DOMAIN
```

Important variables:

| Variable | Purpose |
|---|---|
| `POSTGRES_PASSWORD` | DB password used inside Compose |
| `JWT_SECRET` / `JWT_REFRESH_SECRET` | Auth tokens (long random strings) |
| `CORS_ORIGIN` | Web origin(s), comma-separated |
| `NEXT_PUBLIC_API_URL` | Browser → API base (`https://…`) |
| `NEXT_PUBLIC_WS_URL` | Browser → WS (`wss://…/ws`) |
| `WEB_APP_URL` / `API_PUBLIC_URL` | Paymob redirects + webhook base |
| `WEB_DOMAIN` / `API_DOMAIN` | Only for Compose `--profile tls` (Caddy) |

`NEXT_PUBLIC_*` are baked into the web image at **build** time. Change them → rebuild `web`.

---

## Path A — Zero cost (Oracle Always Free + Docker)

### 1. Create the VM

1. Sign up for Oracle Cloud (Always Free).
2. Create a **VM.Standard.A1.Flex** instance (Arm), e.g. **2 OCPU / 12 GB**, Ubuntu 22.04/24.04.
3. Assign a public IP.
4. Ingress rules: **22**, **80**, **443** (and **3000/4000** only if you skip Caddy).

Capacity for ARM shapes can be limited by region; try nearby regions if create fails.

### 2. Install Docker on the VM

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"
# log out and back in
docker --version
```

### 3. Deploy the app

```
git clone <YOUR_REPO_URL> HRSystem
cd HRSystem
cp .env.production.example .env.production
nano .env.production   # set secrets + YOUR_HOST (or domains)
```

**HTTP first (IP or hostname, ports 3000/4000):**

```
docker compose -f docker-compose.prod.yml --env-file .env.production up -d
--build
```

**HTTPS (recommended):** point DNS A records for `app.example.com` and `api.example.com` at the VM, set `WEB_DOMAIN` / `API_DOMAIN` and the `https` / `wss` URL vars in `.env.production`, then:

```
docker compose -f docker-compose.prod.yml --env-file .env.production --
profile tls up -d --build
```

Caddy obtains Let's Encrypt certificates automatically. Prefer firewalling host ports 3000/4000 when TLS is enabled and only exposing 80/443.

### 4. Seed (once)

```
docker compose -f docker-compose.prod.yml --env-file .env.production exec
api \
  node packages/db/dist/seed.js
```

Change seeded admin passwords immediately (or set `SEED_SUPER_ADMIN_*` before seeding).

### 5. Smoke checks

```
curl -fsS "$API_PUBLIC_URL/health"
# open WEB_APP_URL → login
# admin live map → confirm WebSocket
```

### 6. Paymob (when ready)

Register webhook: `{API_PUBLIC_URL}/v1/webhooks/paymob`
Fill Paymob env vars and recreate the API container. Details: BILLING.md.

### 7. Backups

```
docker compose -f docker-compose.prod.yml --env-file .env.production exec -
T postgres \
  pg_dump -U hr hr_system > backup-$(date +%Y%m%d).sql
```

Schedule this (cron) off-box. Keep a single API replica — hourly jobs run **inside** the API process.

---

# Path B — Best cheap paid (Railway)

Approximate cost: Hobby **$5/mo** plan fee + usage (often **~$10–20/mo** for always-on API + Postgres + web).

## 1. Create project

1. Create a Railway project.
2. Add a **PostgreSQL** plugin; copy `DATABASE_URL`.
3. Add two services from this GitHub repo (or CLI): **api** and **web**.

## 2. API service

- Root directory: repo root
- Dockerfile: `apps/api/Dockerfile`
- Or Nixpacks/build: install with `pnpm`, build shared/db/api, start with migrate then `node apps/api/dist/index.js`
- Set env:

```
DATABASE_URL=<from Railway Postgres>
JWT_SECRET=...
JWT_REFRESH_SECRET=...
API_PORT=4000
API_HOST=0.0.0.0
CORS_ORIGIN=https://your-web-domain
WEB_APP_URL=https://your-web-domain
API_PUBLIC_URL=https://your-api-domain
# optional Paymob vars
```

Generate a public HTTPS domain for the API service. Confirm `/health` and that WebSockets work on `wss://your-api-domain/ws`.

Migrations run automatically when using the Docker entrypoint. Otherwise run once:

```
pnpm --filter @hr/db build && pnpm --filter @hr/db migrate:prod
```

Seed once (Railway shell or one-off):

```
pnpm --filter @hr/db seed:prod
# or: node packages/db/dist/seed.js
```

## 3. Web service

- Dockerfile: `apps/web/Dockerfile`
- Build args / env at **build** time:

```
NEXT_PUBLIC_API_URL=https://your-api-domain
NEXT_PUBLIC_WS_URL=wss://your-api-domain/ws
```

Attach a public domain. Rebuild after changing `NEXT_PUBLIC_*`.

### 4. Alternative split

- **Neon** (or Railway) Postgres + Railway API + **Vercel** web is fine.
- Keep the API on a long-lived host (Railway/Fly/VM), not Vercel serverless.

### 5. Paymob

Same as Path A: `{API_PUBLIC_URL}/v1/webhooks/paymob`.

---

## Local production Compose smoke test

From your laptop (Docker required):

```
cp .env.production.example .env.production
# set POSTGRES_PASSWORD, JWT_*, and localhost URLs:
#   CORS_ORIGIN=http://localhost:3000
#   NEXT_PUBLIC_API_URL=http://localhost:4000
#   NEXT_PUBLIC_WS_URL=ws://localhost:4000/ws
#   WEB_APP_URL=http://localhost:3000
#   API_PUBLIC_URL=http://localhost:4000

docker compose -f docker-compose.prod.yml --env-file .env.production up -d
--build
curl -fsS http://localhost:4000/health
```

Stop:

```
docker compose -f docker-compose.prod.yml --env-file .env.production down
```

---

## Launch checklist

- [ ] Strong `POSTGRES_PASSWORD`, `JWT_SECRET`, `JWT_REFRESH_SECRET`
- [ ] HTTPS in production (`https` / `wss`); CORS matches the web origin
- [ ] Only **one** API replica (in-process jobs)
- [ ] `pnpm`/Docker migrate succeeded (API entrypoint or manual)
- [ ] Seeded admins; default passwords rotated
- [ ] `/health` OK; login OK; live map WS OK
- [ ] Postgres backup plan (`pg_dump` or provider backups)

- [ ] Paymob webhook registered when enabling paid checkout
- [ ] `.env.production` never committed

## Useful commands

```
# logs
docker compose -f docker-compose.prod.yml --env-file .env.production logs -
f api web

# rebuild after code or NEXT_PUBLIC_* changes
docker compose -f docker-compose.prod.yml --env-file .env.production up -d
--build

# shell into API
docker compose -f docker-compose.prod.yml --env-file .env.production exec
api sh
```